

Code Explanation

Delta Live Tables Test Explanation

This test suite is designed to validate the transformations that would be applied in Delta Live Tables (DLT) using PySpark. The test class `TestDeltaLiveTables` contains methods to set up test data, simulate DLT transformations, and verify the results.

Overview of the Code

The code creates a test class `TestDeltaLiveTables` that inherits from `unittest.TestCase`. It sets up a Spark session, creates test data for customers and orders, and writes the data to temporary CSV files. The test method `test\_dlt\_transformations` simulates the transformations that would be applied in DLT, including data cleansing, aggregation, and joining.

Step 1: Setting Up the Test Environment

The `setUpClass` method is used to set up the test environment. It creates a Spark session, defines the test data for customers and orders, and writes the data to temporary CSV files.

```
@classmethod
def setUpClass(cls):
    cls.spark = SparkSession.builder
        .appName("TestDeltaLiveTables")
        .master("local[1]")
        .getOrCreate()

    customer_data = [
        ("C001", "John Doe", "john@example.com", "North"),
        ("C002", "Jane Smith", "jane@example.com", "South"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C003", "Bob Johnson", "bob@example.com", "East"),
        ("C004", None, "alice@example.com", "West"),
        ("C005", "Charlie Brown", "charlie@example.com", "North")
    ]

    customer_schema = StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True)
    ])

    cls.customer_df = cls.spark.createDataFrame(customer_data, schema=customer_schema)
```

Step 2: Creating Test Data and Writing to Temporary Files

The test data is created using PySpark DataFrames, and the data is written to temporary CSV files using the `write.format("csv")` method.

```
cls.customer_df.write.format("csv").option("header", "true").save(cls.customer_path)
cls.order_df.write.format("csv").option("header", "true").save(cls.order_path)
```

Step 3: Simulating Customer Silver Transformation

The `test\_dlt\_transformations` method simulates the customer silver transformation by removing duplicates and null values from the customer DataFrame.

```
customer_silver = self.customer_df
    .dropDuplicates(["CustId"])
    .filter(self.customer_df.CustId.isNotNull() &
            self.customer_df.Name.isNotNull() &
            self.customer_df.EmailId.isNotNull() &
            self.customer_df.Region.isNotNull())
```

Step 4: Simulating Order Silver Transformation

The order silver transformation is simulated by adding a new column `TotalAmount`, removing duplicates, and filtering out null values.

```
order_silver = self.order_df
    .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    .dropDuplicates(["OrderId"])
    .filter(self.order_df.OrderId.isNotNull() &
            self.order_df.ItemName.isNotNull() &
            self.order_df.PricePerUnit.isNotNull() &
            self.order_df.Qty.isNotNull() &
            self.order_df.Date.isNotNull() &
            self.order_df.CustId.isNotNull())
```

Step 5: Verifying the Transformations

The test method verifies the results of the transformations by checking the count of rows and the values in the resulting DataFrames.

```
self.assertEqual(customer_silver.count(), 4)
self.assertEqual(order_silver.count(), 4)
```

Step 6: Simulating Join and Aggregation

The test method simulates a join between the customer silver and order silver DataFrames and performs an aggregation to calculate the total amount spent by each customer.

```
joined_df = customer_silver.join(
    order_silver,
    on="CustId",
    how="inner"
).select(
    customer_silver["CustId"],
    customer_silver["Name"],
    customer_silver["EmailId"],
    customer_silver["Region"],
    order_silver["OrderId"],
    order_silver["ItemName"],
    order_silver["PricePerUnit"],
```

```
order_silver["Qty"],
order_silver["Date"],
order_silver["TotalAmount"]
)
aggregate_df = joined_df.groupBy("Name", "Date")
    .agg(sum ("TotalAmount").alias("TotalAmount"))
```

Step 7: Cleaning Up the Test Environment

The `tearDownClass` method is used to clean up the test environment by stopping the Spark session and deleting the temporary files.

```
@classmethod
def tearDownClass(cls):
    cls.spark.stop()
    shutil.rmtree(cls.temp_dir)
```